ACRME POC Test Workbook — v2.4 (Canonical Spec of Record)

Azure Capacity Reservation Management Engine (ACRME) — POC Validation Runbook & Progress Tracker

Field	Value
Version	2.4 (canonical spec of record)
Status	ACTIVE — aligned to Requirements Baseline v2.4 and the executable `POC-Test-Scripts/`
Supersedes	`acrme_poc_workbook_v2.docx` / `.pdf` (v2.0, 21 Aug 2026 — archived, binary-only)
Authoritative source	`Azure Capacity & Quota Management — Consolidated Requirements Baseline` (v2.4); mirrored at `Requirements/acrme_requirements_baseline_v2_4.md`
Executable implementation	`POC-Test-Scripts/` (groups G1–G12; run `python runner.py` and `python test_v24_reservation_model.py`)
Classification	Internal — Engineering Validation
Owner	[Test Lead Name]
Environment	[Subscription ID / Tenant ID — fill on start]

> **What this document is.** This is the single **POC workbook of record**. It consolidates the manual runbook/tracker content previously held only in the binary `acrme_poc_workbook_v2.docx` / `.pdf` (574 paragraphs, 140 tables) into a Markdown source that follows the repo's md-source convention, and it is kept in lock-step with the executable `POC-Test-Scripts/`. Expected results are hypotheses until executed with retained evidence. Preview features are not Microsoft contractual commitments; Tier 3 and VMSS emergency automation remain blocked in Phase 1.
>
> **v2.4 reconciliation (what changed from v2.0).**
> - **Region model — two-region-aware (PLC-010a).** The former "three distinct regions" hard rule (old §3.1, PF-09/PF-10) is now **geography-aware**: three distinct regions are required **only** for three-region geographies (US is the only one today); **two-region geographies co-locate CVAL + DR in the non-prod region** while the other region hosts Prod (ENV-003 / PLC-010a). **Middle East** is the sole `DR_NOT_OFFERED` case, on legal grounds (DR-014 / DEC-001) — not a general two-region outcome. This mirrors the Phase 1 rework of `POC-Test-Scripts/acrme_suite/config.py` and `pre-flight.py`.
> - **New v2.4 coverage — Groups G9–G12.** Added test groups for CAP-020/CAP-021/CAP-001a, PLC-010a (positive), PLC-011 (even zone distribution + rebalance, reproducing Calc-Logic Scenario 21), CAP-022/CAP-024 (seed-at-0 matrix + reactive discovery), and CAP-023/OPS-006 (regional + per-AZ CRG structure and deterministic naming). Inventory total: **49** cases across **12** groups (was 35 across 8).
> - **Numbering unified** to the executable suite's POC IDs (see §4.3). Where the earlier research workbook used a divergent POC-01…51 scheme, a cross-reference is provided rather than a second numbering.

---

1. How to Use This Workbook

This section defines the operating rules for engineers executing ACRME validation tests. Follow the sequence strictly; later groups depend on earlier evidence.

1. Fill in the Environment Details table (Section 2) before starting any test. Do not proceed until every required field is populated.
2. Complete the Pre-Flight Checklist (Section 3) and mark every item Verified = Y before executing Group 1.
3. Work through POC groups in order. Later groups may depend on earlier ones passing. Prerequisites are listed on each POC entry.
4. Record actual results in the Actual Result field immediately after each test step completes. Do not batch-fill results at the end of a session.
5. Mark Status as Pass, Fail, or Blocked. If Blocked, record the blocker description in Notes / Blocker and open a row in the Issue and Blocker Log (Section 7).
6. All tests use the Azure CLI (az) unless otherwise specified. Ensure you are logged in with az login and have the correct subscription set with az account set –subscription \<SUBSCRIPTION_ID>.
7. REST API calls use az rest syntax for consistency. Always pin the api-version shown in the step. Do not substitute a different version without recording the change.

8. Retain request and response identifiers, timestamps, before/after state, API version, region, SKU, and zone for every executed POC. These are required evidence for phase-gate sign-off.
9. Preview features (CRG Sharing, Quota Groups) are not Microsoft contractual commitments. Record observed behavior as Observed, not as SLA.
10. Do not automate Tier 3 or VMSS emergency operations in Phase 1. Those paths are blocked by design (G-14, FC-08).

1.1 Evidence retention requirements

Each executed POC must retain the following evidence fields (architecture §42):

- API version
- Region
- SKU
- Zone (logical and physical if known)
- VM or VMSS mode
- Request and response identifiers (x-ms-request-id / correlation)
- Timestamps (start, ARM confirm, ARG confirm)
- Before and after state snapshots
- Retry history
- Observed propagation distribution
- Classification: Documented / Observed / Assumed / Judgement

1.2 Status vocabulary

Status	Meaning
[Not Started]	Test has not been executed
[In Progress]	Execution underway; results incomplete
[Pass]	Observed result meets expected criteria for this API version/region/SKU
[Fail]	Observed result contradicts expected criteria; open blocker if needed
[Blocked]	Cannot execute due to dependency, access, or platform unavailability

2. Environment Details

Complete this table before any POC execution. Region placement must conform to the geography's `distribution_model` (see §3.1): Prod is always separated from Non-Prod/CVAL; DR is a distinct region only in **three-region** geographies (US), **co-located with CVAL** in the non-prod region in **two-region** geographies (PLC-010a), and `DR_NOT_OFFERED` for Middle East (DEC-001).

Field	Value (fill at start)
Provider Subscription ID	
Consumer Subscription ID	
Tenant ID	
Resource Group (Provider)	
Resource Group (Consumer)	
Azure Region (Primary)	
Azure Region (DR) — must differ from Primary	
Azure Region (Non-Production) — must differ from Primary and DR	
VM SKU Under Test	
CRG Name (Provider)	
Capacity Reservation Name	
Quota Group Name (if applicable)	
Test Engineer Name	
Test Start Date	
Test Completion Target Date	

2.1 Naming conventions (recommended)

Resource	Pattern	Example
Provider RG	rg-acrme-provider-\<region>	rg-acrme-provider-west-europe
Consumer RG	rg-acrme-consumer-\<region>	rg-acrme-consumer-west-europe
Prod CRG	crg-prod-\<region>-\<sku>	crg-prod-westeurope-d4s
NonProd CRG	crg-nonprod-\<region>-\<sku>	crg-nonprod-northeurope-d4s
DR CRG	crg-dr-\<region>-\<sku>	crg-dr-uksouth-d4s
Capacity Reservation	cr-\<env>-\<sku>-z\<zone>	cr-prod-d4s-z1
Test VM	vm-poc-\<id>-\<n>	vm-poc-03-01

3. Pre-Flight Checklist

Every item must be Verified = Y before Group 1 begins. Paste command output into Notes where useful.

Item	Requirement	Command	Verified Y/N	Notes
PF-01	Azure CLI version >= 2.50.0	az –version		
PF-02	Logged in with correct identity	az account show		
PF-03	Correct subscription is active	az account set –subscription \<PROVIDER_SUB> && az account show –query id -o tsv		
PF-04	Required resource providers registered: Microsoft.Compute, Microsoft.Quota	az provider list –query “[?registrationState==’Registered’].namespace” -o tsv | grep -E ‘Microsoft\.(Compute|Quota)’		
PF-05	Provider subscription has sufficient quota for the SKU under test	az vm list-usage –location \<PRIMARY_REGION> –query “[?contains(name.value, ‘standardDSv3Family’) || contains(name.value, ‘\<SKU_FAMILY>‘)]” -o table		
PF-06	Consumer subscription has sufficient quota for the SKU under test	az account set –subscription \<CONSUMER_SUB> && az vm list-usage –location \<PRIMARY_REGION> –query “[?con-		

Item	Requirement	Command	Verified Y/N	Notes
		tains(name.value, '\<SKU_FAMILY>')]" -o table		
PF-07	RBAC role verified on provider CRG scope (after CRG exists) or planned role assignments documented	az role assignment list –scope \<CRG_RESOURCE_ID> -o table		
PF-08	No existing CRGs that could conflict with test names	az capacity reservation group list –resource-group \<PROVIDER_RG> -o table		
PF-09	**Geography-aware** region model confirmed for the geography under test (see §3.1). Prod region ≠ non-prod/CVAL region in **all** geographies.	echo Prod=\<PRIMARY_REGION> NonProd=\<NONPROD_REGION>; test "\<PRIMARY_REGION>" != "\<NONPROD_REGION>" && echo PASS || echo FAIL		
PF-10	DR placement confirmed per `distribution_model`: **three-region** geographies (US) → DR region distinct from both Prod and non-prod; **two-region** geographies →	echo Model=\<DISTRIBUTION_MODEL> Prod=\<PRIMARY_REGION> NonProd=\<NONPROD_REGION> DR=\<DR_REGION>; # three-region: DR != Prod && DR !=		

Item	Requirement	Command	Verified Y/N	Notes
	DR **co-located** with CVAL in the non-prod region (PLC-010a); **Middle East** → `DR_NOT_OFFERED` (DEC-001).	NonProd; two-region: DR == NonProd; middle-east: DR == NOT_OFFERED		

3.1 Geography-aware region placement check (v2.4)

> **v2.4 change.** The former “three distinct regions” absolute rule is **superseded**. Region separation is now **geography-aware**, driven by each geography’s `distribution_model` (PLC-010a / ENV-003). Production is always separated from non-prod/CVAL; DR placement depends on how many Standard regions the geography offers.

Rules by `distribution_model`:

- **Three-region geography** (today **US** is the only one): Prod, Non-Prod/CVAL, and DR each in a **distinct** region. All three of Prod≠NonProd, DR≠Prod, DR≠NonProd must hold.
- **Two-region geography** (e.g. EU with Switzerland North + Sweden Central; APAC pairs): one region hosts **Prod**; the other region hosts **CVAL + DR co-located** (PLC-010a). Here **DR == NonProd/CVAL region is valid and expected** — it is not a violation. Prod≠NonProd still holds.
- **Middle East**: `DR_NOT_OFFERED` — DR is not placed in-geo, on legal grounds (DR-014 / DEC-001). This is Middle-East-specific and must **not** be generalised to other two-region geographies.

PF-09/PF-10 are blocking against the **applicable** rule above. Record the geography model and regions below and obtain DR Architect countersignature.

Role	Region name	Conforms to `distribution_model`?	Sign-off
Geography / `distribution_model` (three-region / two-region / DR_NOT_OFFERED)		Y / N	
Prod (Primary)		Y / N	
Non-Prod / CVAL		Y / N	
DR (or `NOT_OFFERED`)		Y / N	

Reference: this mirrors the Phase 1 rework of `POC-Test-Scripts/acrme_suite/config.py` (per-geography `distribution_model`) and `preflight.py` (geography-aware PF-09/PF-10), and the positive two-region case POC-PLC-010a in Group 9.

4. POC Test Groups

Execute groups in order unless a later group explicitly lists only Pre-Flight prerequisites. Phase gates are indicated on each POC. Production-phase tests (for example POC-10) must not run against pilot customer resources.

4.0 Test inventory summary

Group	Name	POC count
G1	CRG Creation and Basic Reservation	5
G2	Cross-Subscription Sharing (Preview Feature)	6
G3	DR Capacity and Failover	4
G4	Quota Group Validation (Preview — Gated)	3
G5	Engine Safety Controls	6
G6	AKS and VMSS Behaviour	6
G7	API Rate and Throttle Behaviour (FC-16)	3
G8	Reserved Instance Discount Scope (FC-09)	2
G9	Reservation Eligibility & Two-Region Model (CAP-020/021/001a, PLC-010a)	5
G10	Even Zone Distribution & Rebalance (PLC-011, A.9, C-13)	3
G11	Seed Matrix & Reactive Discovery (CAP-022/024)	3
G12	Regional + Per-AZ CRG & Deterministic Naming (CAP-023, OPS-006/C-12)	3
TOTAL	All groups	49

> **G9–G12 are the v2.4 additions.** Each is implemented in `POC-Test-Scripts/acrme_suite/tests/g9_reservation_eligibility.py`, `g10_zone_distribution.py`, `g11_seed_matrix.py`, `g12_naming_convention.py`, plus the offline pytest `POC-Test-Scripts/test_v24_reservation_model.py`. Pure-logic cases execute **offline** (no Azure); the `*-LIVE` cases return **BLOCKED** until the engine/preview APIs are available (consistent with POC-18/19/20).

4.0.1 Recommended execution order

Follow this order unless a blocker forces a documented deviation. Deviations must be recorded in §7.

Seq	Work package	Exit criteria
0	Groups G9–G12 offline logic (`python test_v24_reservation_model.py`)	v2.4 reservation-model logic proven before any Azure spend
1	Pre-Flight PF-01…PF-10	Environment ready; geography-aware region model proven (§3.1)
2	Group 1 POC-01…POC-05	Provider CRG and basic reservation lifecycle
3	Group 2 POC-06, POC-06a, POC-07…POC-09	Sharing + zone gate before consumer load
4	Group 4 POC-30	Hard gate for quota-group engineering
5	Group 5 POC-15, POC-16, POC-17, POC-20	Safety hypotheses and Tier 3 rejection
6	Group 7 POC-THROTTLE-01…03	Throttle baselines before automation concurrency
7	Group 8 POC-RI-01…02	FinOps before customer cost models
8	Group 3 POC-11, POC-13 (then 12/14 in Phase 2)	DR capacity and floor accounting
9	Group 6 POC-AKS-02 early; AKS/VMSS create paths in Phase 2	Disruption and preview limits
10	Group 5 POC-18/19 and Group 4 POC-31/32	Only after engine + quota gates
11	POC-10 and Production gates	Isolated environment; leadership acceptance

Group 1: CRG Creation and Basic Reservation

Validates foundational Capacity Reservation Group and Capacity Reservation lifecycle in the provider subscription. All subsequent groups depend on a healthy provider CRG and at least one reservation.

Group contains 5 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-01: Create CRG in Provider Subscription

Attribute	Value
POC ID	POC-01
Group	Group 1: CRG Creation and Basic Reservation
Objective	Create a zonal Capacity Reservation Group in the provider subscription and confirm provisioning succeeds with the requested zones.
Phase Gate	Phase 1 Pilot
Prerequisites	Pre-Flight Checklist complete (PF-01 through PF-10)
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Set provider context:

az account set –subscription \<PROVIDER_SUB>

2. Create CRG:

az capacity reservation group create –resource-group \<PROVIDER_RG> –name \<CRG_NAME> –location \<PRIMARY_REGION> –zones 1 2 3

3. Verify:

az capacity reservation group show –resource-group \<PROVIDER_RG> –name \<CRG_NAME> -o json

Expected result

provisioningState = Succeeded; zones array populated with requested zone IDs; location matches PRIMARY_REGION. Record full resource ID for later steps.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-02: Create Capacity Reservation within the CRG

Attribute	Value
POC ID	POC-02
Group	Group 1: CRG Creation and Basic Reservation
Objective	Create a Capacity Reservation for the target VM SKU and quantity inside the provider CRG.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-01 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Create reservation:

az capacity reservation create –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –sku \<VM_SKU> –capacity \<QUANTITY> –location \<PRIMARY_REGION>

2. Optional zone pin: add –zone \<ZONE> if testing zonal reservation

3. Verify:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> -o json

Expected result

provisioningState = Succeeded; sku.name = VM_SKU; capacity (reserved quantity) = QUANTITY. Record reservedCapacity and instanceView if present.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-03: Associate VM with Reservation (Provider Subscription)

Attribute	Value
POC ID	POC-03
Group	Group 1: CRG Creation and Basic Reservation
Objective	Deploy a VM in the provider subscription associated to the CRG and confirm the association is recorded on the VM resource.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-02 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Create VM:

az vm create –resource-group \<PROVIDER_RG> –name \<VM_NAME> –image Ubuntu2204 –size \<VM_SKU> –capacity-reservation-group /subscriptions/\<PROVIDER_SUB>/resourceGroups/\<PROVIDER_RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG_NAME> –generate-ssh-keys –location \<PRIMARY_REGION>

2. If zonal CR: add –zone \<ZONE> matching the reservation zone

3. Verify association:

az vm show –resource-group \<PROVIDER_RG> –name \<VM_NAME> –query capacityReservation -o json

Expected result

VM provisioningState = Succeeded; capacityReservation.capacityReservationGroup.id matches the provider CRG resource ID.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-04: Verify Capacity Consumption Count Increments

Attribute	Value
POC ID	POC-04
Group	Group 1: CRG Creation and Basic Reservation
Objective	Confirm that associating a VM increments virtualMachinesAssociated on the Capacity Reservation without changing reserved quantity.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-03 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Query associations:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query virtualMachinesAssociated -o json

2. Query reserved quantity:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query '{sku:sku, capacity:sku.capacity, provisioning-State:provisioningState}' -o json

Expected result

virtualMachinesAssociated count = 1 (or previous + 1); reserved capacity unchanged from POC-02.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		

POC-05: Disassociate VM — Verify Capacity Released

Attribute	Value
POC ID	POC-05
Group	Group 1: CRG Creation and Basic Reservation
Objective	Disassociate a running/deallocated VM from the CRG and confirm association count decrements while reservation quantity remains.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-04 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Deallocate VM:

az vm deallocate –resource-group \<PROVIDER_RG> –name \<VM_NAME>

2. Remove CRG association:

az vm update –resource-group \<PROVIDER_RG> –name \<VM_NAME> –set capacityReservation.capacityReservationGroup=null

3. Alternative CLI form (if supported in your CLI build):

az vm update –resource-group \<PROVIDER_RG> –name \<VM_NAME> –capacity-reservation-group None

4. Verify count:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query virtualMachinesAssociated -o json

5. Verify VM association cleared:

az vm show –resource-group \<PROVIDER_RG> –name \<VM_NAME> –query capacityReservation -o json

Expected result

Association count decrements; reservation quantity unchanged; VM no longer references the CRG. Record exact CLI form that succeeded.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

G1 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 2: Cross-Subscription Sharing (Preview Feature)

Validates CRG sharingProfile across subscriptions. Sharing is a Preview dependency (A-01, R-01). Always pin api-version=2024-03-01 or the preview version that succeeds, and record the exact version. Zone alignment (POC-06a / FC-06) is an onboarding gate before any customer is onboarded.

Group contains 6 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-06: Enable Cross-Subscription Sharing on CRG

Attribute	Value
POC ID	POC-06
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Add the consumer subscription to the provider CRG sharingProfile via REST and confirm read-back.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-02 Pass. NOTE: Preview feature — record exact API version used.
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<PROVIDER_SUB>

2. Share CRG:

az rest –method PATCH –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/resourceGroups/\<PROVIDER_RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG_NAME>?api-version=2024-03-01’ –body ‘{“properties”:{“sharingProfile”:{“subscriptionIds”:[{“id”:”/subscriptions/\<CONSUMER_SUB>“}]}}}’

3. If 2024-03-01 fails, retry with api-version=2024-03-01-preview and record which version succeeded

4. Verify:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/resource-Groups/\<PROVIDER_RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG_NAME>?api-version=2024-03-01’ –query properties.sharingProfile -o json

Expected result

sharingProfile.subscriptionIds contains /subscriptions/\<CONSUMER_SUB>; provisioningState = Succeeded. Record: exact API version that succeeded; region tested.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-06a: Cross-Subscription Zone Alignment Validation (FC-06 — Critical)

Attribute	Value
POC ID	POC-06a
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Confirm logical-to-physical availability zone mappings for provider and consumer subscriptions. Zone mismatch means the reservation is in the wrong physical zone for the consumer. This test must pass before any customer is onboarded.
Phase Gate	Phase 1 Pilot — ONBOARDING GATE
Prerequisites	POC-06 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Get provider zone mappings:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/locations?api-version=2022-12-01’ –query “[?name==’\<PRIMARY_REGION>‘].availabilityZoneMappings” -o json

2. Get consumer zone mappings:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<CONSUMER_SUB>/locations?api-version=2022-12-01’ –query “[?name==’\<PRIMARY_REGION>‘].availabilityZoneMappings” -o json

3. Build comparison table: for each logical zone label (1, 2, 3), record provider physical zone name and consumer physical zone name

4. Determine whether logical zone used by the CRG maps to the same physical zone on the consumer subscription

5. If misaligned: identify the consumer logical zone that maps to the provider CRG physical zone (zone translation algorithm)

Expected result

Full zone mapping table recorded for both subscriptions. Zones must align for the CRG zone and the consumer intended deployment zone, OR the translation mapping must be documented. If no mapping exists for the physical zone: BLOCKED — escalate to engineering immediately (ZoneMappingUnavailable).

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-07: Consumer Discovers Shared CRG via ARG

Attribute	Value
POC ID	POC-07
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Document that consumer-side standard list may omit shared CRGs (known Microsoft issue R-03) and that Azure Resource Graph returns the shared CRG for diagnostics.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-06 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<CONSUMER_SUB>

2. ARG query:

az graph query -q “Resources | where type =\~ ‘microsoft.compute/capacityreservationgroups’ | where id startswith ‘/subscriptions/\<PROVIDER_SUB>’ | project id, name, location, properties” -o json

3. Direct list (expected incomplete):

az capacity reservation group list –subscription \<CONSUMER_SUB> -o table

4. Optional REST list:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<CONSUMER_SUB>/providers/Microsoft.Compute/capacityReservationGroups?api-version=2024-03-01’ -o json

Expected result

ARG returns the shared provider CRG. Direct list omits it or returns empty when the consumer has no local CRG. This documents the known Microsoft bug — it is not a test failure. Record both outputs.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-08: Associate Consumer VM with Shared CRG

Attribute	Value
POC ID	POC-08
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Deploy a VM in the consumer subscription against the shared provider CRG using the correct consumer logical zone from POC-06a.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-07 Pass; POC-06a Pass (zone alignment or translation documented)
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<CONSUMER_SUB>

2. Create consumer VM using translated zone if required:

az vm create –resource-group \<CONSUMER_RG> –name \<CONSUMER_VM_NAME> –image Ubuntu2204 –size \<VM_SKU> –capacity-reservation-group /subscriptions/\<PROVIDER_SUB>/resourceGroups/\<PROVIDER_RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG_NAME> –generate-ssh-keys –location \<PRIMARY_REGION> –zone \<CONSUMER_LOGICAL_ZONE>

3. Verify association:

az vm show –resource-group \<CONSUMER_RG> –name \<CONSUMER_VM_NAME> –query capacityReservation -o json

Expected result

VM created successfully; capacityReservation.capacityReservationGroup.id equals the provider CRG resource ID. If placement fails, capture error and re-check zone translation (FC-06 / R-41).

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-09: Verify Combined Consumption Count

Attribute	Value
POC ID	POC-09
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Confirm provider-side reservation association count reflects both provider and consumer VMs.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-08 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<PROVIDER_SUB>

2. Run:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query virtualMachinesAssociated -o json

3. Optionally re-associate a provider VM (POC-03 pattern) and re-check combined count

Expected result

Count reflects all associated VMs across provider and consumer subscriptions. Record association resource IDs.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-10: 100-Consumer Limit Boundary Behaviour

Attribute	Value
POC ID	POC-10
Group	Group 2: Cross-Subscription Sharing (Preview Feature)
Objective	Observe API behaviour near the documented 100-consumer subscription limit per CRG (A-02, R-02). Isolated test environment only.
Phase Gate	Production
Prerequisites	POC-09 Pass. WARNING: Run in isolated test environment only. Do not run against pilot customer CRGs.
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. From a controlled script or sequential PATCH calls, add consumer subscription IDs to sharingProfile until the API returns an error

2. Record the exact error message, status code, and the count at which the error was triggered

3. Verify read-back of sharingProfile after the failure

4. Clean up: remove test consumer entries from sharingProfile immediately after the test

Expected result

API returns an error near the 100-consumer limit. Document exact limit observed, error code, and error message. Enforce conservatively in engine design.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

G2 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 3: DR Capacity and Failover

Validates DR-region CRG pre-positioning, simulated failover, engine-enforced DR floor accounting, and failback. Prod, NonProd, and DR must not all share one region. DR floor is an engine control, not a native Azure sub-reservation (A-06 rejected).

Group contains 4 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-11: Create DR Region CRG and Reservation

Attribute	Value
POC ID	POC-11
Group	Group 3: DR Capacity and Failover
Objective	Create CRG and Capacity Reservation in the DR region, confirming location isolation from Primary and NonProd.
Phase Gate	Phase 1 Pilot
Prerequisites	Pre-flight PF-09 and PF-10 Pass; POC-02 pattern understood
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<PROVIDER_SUB>

2. Create DR CRG:

az capacity reservation group create –resource-group \<PROVIDER_RG> –name \<DR_CRG_NAME> –location \<DR_REGION> –zones 1 2 3

3. Create DR reservation:

az capacity reservation create –resource-group \<PROVIDER_RG> –capacity-reservation-group \<DR_CRG_NAME> –name \<DR_CR_NAME> –sku \<VM_SKU> –capacity \<DR_QUANTITY> –location \<DR_REGION>

4. Verify locations:

az capacity reservation group show –resource-group \<PROVIDER_RG> –name \<DR_CRG_NAME> –query location -o tsv

Expected result

CRG and reservation created in DR_REGION; location in response confirms DR region, not Primary or NonProd.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-12: Simulate DR Failover

Attribute	Value
POC ID	POC-12
Group	Group 3: DR Capacity and Failover
Objective	Simulate failover by deallocating primary workload capacity path and deploying a DR VM against the DR CRG. Record elapsed times.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-11 Pass; sharing to DR consumer configured if cross-sub
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Record T0 wall-clock time

2. Deallocate primary VM (if testing release path):

az vm deallocate –resource-group \<PROVIDER_RG> –name \<VM_NAME>

3. Record deallocate completion time T1

4. Create DR VM against DR CRG:

az vm create –resource-group \<DR_RG_OR_PROVIDER_RG> –name \<DR_VM_NAME> –image Ubuntu2204 –size \<VM_SKU> –capacity-reservation-group /subscriptions/\<PROVIDER_SUB>/resourceGroups/\<PROVIDER_RG>/providers/Microsoft.Compute/capacityReservationGroups/\<DR_CRG_NAME> –generate-ssh-keys –location \<DR_REGION>

5. Record time to Running state T2

6. Verify DR association and reservation consumption

Expected result

DR VM reaches Running; capacity consumed from DR reservation. Record total elapsed time (T2-T0) and component times. This is not an application RTO proof.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		
6		

POC-13: DR Floor Protection Verification

Attribute	Value
POC ID	POC-13
Group	Group 3: DR Capacity and Failover
Objective	Calculate and verify the engine DR floor accounting: NonProd must not consume below the protected DR floor within the shared NonProd+DR quota group model.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-11 Pass; potential_dr_demand and DR_RATIO_MAX known for the scenario
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Record DR reserved quantity:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<DR_CRG_NAME> –name \<DR_CR_NAME> –query sku.capacity -o tsv

2. Calculate DR_Floor_vCPU = Potential_DR_Demand × vCPU_Per_Instance × DR_Ratio_Max (default DR_Ratio_Max = 0.40)

3. Calculate Effective_NonProd_Ceiling = NonProd_DR_Group_Limit - DR_Floor_vCPU

4. Record current NonProd used vCPU and confirm NonProd_Used + DR_Floor \<= Group_Limit

5. Document that Azure does not natively enforce this floor — engine detector must protect it (A-06)

Expected result

DR floor calculated and recorded; NonProd allocation respects floor in the test scenario; confirmation that floor is engine-enforced only.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-14: Failback — Restore Primary, Release DR

Attribute	Value
POC ID	POC-14
Group	Group 3: DR Capacity and Failover
Objective	Reverse failover: restore primary path, disassociate or deallocate DR VM, and confirm DR capacity returns to available pool.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-12 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Start or recreate primary VM path and confirm healthy

2. Deallocate DR VM:

az vm deallocate –resource-group \<DR_RG_OR_PROVIDER_RG> –name \<DR_VM_NAME>

3. Disassociate DR VM from DR CRG if required

4. Verify DR reservation associations and available capacity

5. Record time for DR capacity to return to available pool and final state of primary and DR reservations

Expected result

Primary path restored; DR capacity available again; both reservation states recorded. Failback must not destroy the only healthy instance before primary validation (Runbook E).

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

G3 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 4: Quota Group Validation (Preview — Gated)

Quota Groups underpin the two-group architecture (Prod | NonProd+DR). POC-30 is a hard gate before any quota-group engineering begins (B-1, A-04). groupType enforcement is preview-only (FC-11). No fixed propagation SLA is assumed (B-6).

Group contains 3 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-30: Verify Quota Group API Availability

Attribute	Value
POC ID	POC-30
Group	Group 4: Quota Group Validation (Preview — Gated)
Objective	Establish whether the Quota Groups API is available in the target subscription and region with a pinned preview API version. Hard gate for all quota-group engineering.
Phase Gate	Phase 1 GATE — must pass before any quota group engineering begins
Prerequisites	Pre-Flight complete; Microsoft.Quota provider registered
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az account set –subscription \<PROVIDER_SUB>

2. List quota groups:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/providers/Microsoft.Quota/groupQuotas?api-version=2025-03-01-preview’ -o json

3. If endpoint path differs in your cloud, try documented alternatives and record exact URL that responds

4. Attempt creation:

az rest –method PUT –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/providers/Microsoft.Quota/groupQuotas/\<GROUP_NAME>?api-version=2025-03-01-preview’ –body ‘{“properties”:{“displayName”:”ACRME-Test-Group”}}’

5. Repeat list/create probes in Primary, DR, and NonProd regions as applicable to scope

6. Record whether groupType property is present and accepted (FC-11)

Expected result

API returns 200/201 and feature is available. If 404 or MethodNotAllowed: BLOCKED — quota groups not available in this scope; all quota-group engineering is blocked. Record: API version that succeeded; regions tested; result per region.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		
6		

POC-31: Add Subscription to Quota Group and Verify Headroom

Attribute	Value
POC ID	POC-31
Group	Group 4: Quota Group Validation (Preview — Gated)
Objective	Add a subscription to a quota group and measure quota visibility and headroom before vs after membership. Dual subscription-level checks remain mandatory (A-05 rejected).
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-30 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Record baseline subscription quota:

az vm list-usage –location \<PRIMARY_REGION> -o json

2. Add subscription to group via the PUT/PATCH membership API confirmed in POC-30 (record exact URL and body)

3. Poll membership and quota views every 30–60 seconds

4. Compare ungrouped vs grouped quota observations

5. Confirm subscription-level eligibility check is still required before any CR mutation

Expected result

Membership succeeds. Record: quota before group membership; quota after; propagation delay observed. Do not treat membership as proof that deploy will succeed.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-32: Release and Reuse Behaviour

Attribute	Value
POC ID	POC-32
Group	Group 4: Quota Group Validation (Preview — Gated)
Objective	Remove a subscription from a quota group, observe cleanup/propagation, re-add, and confirm consistent behaviour (B-2, A-07 related).
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-31 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Remove subscription from group via REST DELETE or PATCH as confirmed in POC-30

2. Poll until membership cleared; record propagation time

3. Check for orphaned state in group usage views

4. Re-add subscription; verify quota state is consistent with first-add behaviour

5. If testing NonProd reduction funding DR expansion: reduce NonProd CR, poll group headroom, then attempt DR expand only after authoritative headroom is visible

Expected result

Propagation time documented; no unexplained orphaned state; re-add consistent. Tier 2 remains blocked until this behaviour is measured and approved.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

G4 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 5: Engine Safety Controls

Validates safety hypotheses and engine control gates: zero-quantity behaviour (A-08), ARG staleness (A-14), concurrent association, and Tier 1/2/3 controls. Tier 3 remains blocked in Phase 1 (G-14).

Group contains 6 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-15: Zero-Quantity Reservation Behaviour

Attribute	Value
POC ID	POC-15
Group	Group 5: Engine Safety Controls
Objective	Hypothesis validation: observe platform behaviour when reserved capacity is reduced to zero while a VM is or was associated (Path B related). Any result is valid; record precisely.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-03 Pass (VM associated) or controlled re-association. WARNING: May affect running VM guarantees.
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Confirm current association and capacity:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> -o json

2. Attempt zero capacity:

az capacity reservation update –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –capacity 0

3. Check VM state:

az vm show –resource-group \<PROVIDER_RG> –name \<VM_NAME> –query ‘{provisioning-State:provisioningState, powerState:powerState, capacityReservation:capacityReservation}’ -o json

4. Check reservation state after update

5. Restore capacity to a safe non-zero value after evidence capture unless intentionally leaving zero for Path B sequence

Expected result

Document: Does API accept zero-quantity? Does VM remain running? Does VM lose reservation guarantee? Any error? Classify result as Observed for this API version/region/SKU only — not a general guarantee (A-08).

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-16: ARG Indexing Delay Measurement

Attribute	Value
POC ID	POC-16
Group	Group 5: Engine Safety Controls
Objective	Measure ARG lag versus direct ARM reads after a CRG/CR association change. Results set ARG-staleness thresholds. ARG must not confirm destructive mutations (A-14).
Phase Gate	Phase 1 Pilot
Prerequisites	POC-04 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Perform a controlled change (e.g., associate or disassociate a VM) and record ARM confirmation timestamp T_arm

2. Poll ARM:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query virtualMachinesAssociated -o json

3. Poll ARG every 30 seconds for up to 10 minutes:

az graph query -q “Resources | where type =\~ ‘microsoft.compute/capacityreservationgroups’ | where name == ‘\<CRG_NAME>’ | project id, name, properties” -o json

4. Record timestamp T_arg when ARG matches ARM

5. Repeat the measurement 10 times; compute min, median, max delay (T_arg - T_arm)

Expected result

Delay distribution documented (min/median/max). Use results to configure engine ARG-staleness policy. Confirm engine uses ARM for mutation confirmation.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-17: Concurrent Association Safety

Attribute	Value
POC ID	POC-17
Group	Group 5: Engine Safety Controls
Objective	Submit two concurrent VM creates against the same CRG and verify both succeed without association count corruption (related to placement race B-7 at platform layer).
Phase Gate	Phase 1 Pilot
Prerequisites	POC-06 Pass; sufficient reserved capacity for two VMs
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Open two parallel shells with consumer or provider context as designed

2. Simultaneously run two az vm create commands targeting the same CRG resource ID with distinct VM names

3. Wait for both to complete; capture exit codes and error bodies

4. Check count:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –query virtualMachinesAssociated -o json

5. Verify each VM association individually

Expected result

Both VMs associated; count matches successful creates; no silent count mismatch. Record any partial failure for engine hold-design input.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-18: Tier 1 Automatic Capacity Increase

Attribute	Value
POC ID	POC-18
Group	Group 5: Engine Safety Controls
Objective	Validate Tier 1 direct DR expansion path through the engine when DR_EVENT_ACTIVE and headroom checks pass. Phase 2 automation; Phase 1 remains approval-gated for auto-increase (A-15).
Phase Gate	Phase 2 Controlled Automation
Prerequisites	Engine deployed with mode gate implemented (G-15); DR_EVENT_ACTIVE can be declared in test; POC-11 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Confirm engine mode is DR_EVENT_ACTIVE via engine API GET /dr/incidents or equivalent

2. Submit Tier 1 increase: POST /capacity/emergency-transfer (or /capacity/increase-requests) with tier=1, idempotency key, incident ID

3. Poll GET /operations/{id} until terminal state

4. Verify Azure quantity:

az capacity reservation show –resource-group \<PROVIDER_RG> –capacity-reservation-group \<DR_CRG_NAME> –name \<DR_CR_NAME> –query sku.capacity -o tsv

5. Verify audit log entry in engine store with timestamp and operation ID

Expected result

Increase processed per phase policy; quantity updated; audit entry present. If Phase 1 build: request remains approval-gated rather than fully automatic.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-19: Tier 2 Approval Gate

Attribute	Value
POC ID	POC-19
Group	Group 5: Engine Safety Controls
Objective	Confirm Tier 2 quota-neutral transfer requires approval and does not auto-process. Tier 2 disabled until POC-31/32 pass.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	Engine deployed with approval workflow; POC-31 and POC-32 Pass; feature flag for Tier 2 enabled in test only
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Submit Tier 2 operation through engine API with incident ID and impact preview

2. Wait 5 minutes; verify operation remains Pending (not auto-processed)

3. Submit approval through the approval mechanism

4. Poll operation to completion

5. Verify NonProd reduction and DR expansion only after authoritative quota headroom visible; no fixed sleep-based assumption

Expected result

Time in pending state recorded; approval required; completion only after approval; NonProd assurance impact recorded.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

POC-20: Tier 3 Rejection in Phase 1

Attribute	Value
POC ID	POC-20
Group	Group 5: Engine Safety Controls
Objective	Prove Phase 1 engine rejects Tier 3 destructive VM disassociation requests with no partial Azure mutation (G-14 blocker).
Phase Gate	Phase 1 Pilot
Prerequisites	Engine deployed in Phase 1 mode with Tier 3 disabled
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Submit a Tier 3 emergency transfer including a vm_disassociation_list to the engine API

2. Capture rejection response body and HTTP status

3. Verify no CR quantity change and no VM association change occurred in Azure

4. Verify rejection is logged in the audit trail (Tier3AttemptBlocked)

5. Optionally submit a VMSS target and confirm VMSSEmergencyAttempt rejection

Expected result

Clear rejection requiring manual intervention / future enablement; no capacity or association changes; rejection logged.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

G5 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 6: AKS and VMSS Behaviour

Validates workload integration. AKS existing node-pool CRG change requires recreation (FC-18). VMSS Uniform and Flexible must be tested separately. VMSS reprovisioning via shared CRG during zone outage is a Preview limitation (FC-08, R-42). Tier 3 VMSS is blocked in Phase 1.

Group contains 6 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-AKS-01: AKS Node Pool CRG Association at Creation

Attribute	Value
POC ID	POC-AKS-01
Group	Group 6: AKS and VMSS Behaviour
Objective	Create an AKS node pool with a CRG reference and confirm capacityReservationGroupId is set.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-08 Pass (cross-sub sharing confirmed if applicable); AKS cluster available; zone/SKU/quota checks passed
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az aks nodepool add –resource-group \<RG> –cluster-name \<CLUSTER_NAME> –name \<NODEPOOL_NAME> –node-vm-size \<VM_SKU> –capacity-reservation-group \<CRG_RESOURCE_ID> –node-count 1

2. Verify:

az aks nodepool show –resource-group \<RG> –cluster-name \<CLUSTER_NAME> –name \<NODEPOOL_NAME> –query capacityReservationGroupId -o tsv

3. Confirm nodes schedule and reservation associations increment as expected

Expected result

Node pool created; capacityReservationGroupId equals CRG resource ID.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-AKS-02: Verify Existing Node Pool Requires Recreation for CRG Change (FC-18)

Attribute	Value
POC ID	POC-AKS-02
Group	Group 6: AKS and VMSS Behaviour
Objective	Document that changing CRG on an existing node pool is not an in-place update of existing nodes (AKS Node Disruption Policy).
Phase Gate	Phase 1 Pilot
Prerequisites	POC-AKS-01 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Attempt update:

az aks nodepool update –resource-group \<RG> –cluster-name \<CLUSTER_NAME> –name \<NODE-POOL_NAME> –capacity-reservation-group \<NEW_CRG_RESOURCE_ID>

2. If CLI rejects the parameter on update, record the error — that is a valid observation

3. Observe whether API accepts the change; whether existing nodes update in-place or require reimage/recreation

4. Record impact on running workloads

Expected result

Per documentation: in-place update of existing nodes is not supported; recreation/reimage required. Document exact behaviour observed, API response, and disruption.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-VMSS-01: VMSS Uniform CRG Association

Attribute	Value
POC ID	POC-VMSS-01
Group	Group 6: AKS and VMSS Behaviour
Objective	Create a Uniform orchestration VMSS with CRG reference and observe instance association behaviour.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-06 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Create Uniform VMSS with capacity reservation group reference via az vmss create (or REST/template) including –capacity-reservation-group \<CRG_RESOURCE_ID> and –orchestration-mode Uniform

2. Verify model shows CRG reference

3. Scale out one instance if needed; verify instances inherit association

4. Record whether all instances associate immediately or require upgrade; note upgrade policy

Expected result

Model references CRG; instance association behaviour documented for Uniform mode.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-VMSS-02: VMSS Flexible CRG Association

Attribute	Value
POC ID	POC-VMSS-02
Group	Group 6: AKS and VMSS Behaviour
Objective	Repeat CRG association validation for Flexible orchestration mode and capture differences vs Uniform.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-VMSS-01 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Create Flexible VMSS with CRG reference (–orchestration-mode Flexible)

2. Verify model and instance-level association semantics

3. Compare behavioural differences to Uniform results

Expected result

Flexible behaviour documented; differences vs Uniform recorded; no assumption that modes are equivalent.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-VMSS-03: VMSS Disassociation Behaviour

Attribute	Value
POC ID	POC-VMSS-03
Group	Group 6: AKS and VMSS Behaviour
Objective	Remove CRG reference from VMSS model and observe per-instance association state. Manual only in Phase 1 for emergency paths.
Phase Gate	Phase 2 Controlled Automation
Prerequisites	POC-VMSS-01 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Remove CRG reference from VMSS model via update/PATCH

2. Check each instance association state

3. Determine whether reimage, redeploy, or rolling upgrade is required

4. Confirm instances continue running; capture unexpected errors

Expected result

Disassociation behaviour documented per mode. Engine must not claim model change immediately clears all instances without proof.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-VMSS-DR: VMSS Zone Outage via Shared CRG (Preview Limitation — FC-08)

Attribute	Value
POC ID	POC-VMSS-DR
Group	Group 6: AKS and VMSS Behaviour
Objective	Document shared-CRG VMSS reprovisioning behaviour during a simulated zone constraint. Known Preview limitation — failure is acceptable and must be recorded before Phase 2 VMSS DR design.
Phase Gate	Phase 2 REQUIRED BEFORE DR FLOWS
Prerequisites	POC-VMSS-02 Pass; shared CRG configured
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Configure VMSS in the primary zone against shared CRG

2. Simulate zone unavailability constraint (restrict scale/reprovision attempts to DR zone only as test design allows)

3. Attempt to reprovision VMSS instances in DR zone using the shared CRG path

4. Record whether reprovisioning succeeds; exact error if it fails; API version; timestamp; region

5. If failed: document non-shared-CRG fallback requirement for Phase 2

Expected result

Result documented — pass or fail both valid. This test must be run before Phase 2 VMSS DR flows are designed. Pilot customers using VMSS DR must accept the limitation if unresolved.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		
5		

G6 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 7: API Rate and Throttle Behaviour (FC-16)

Documents Compute throttle baselines and observed 429 behaviour. Documented starting baselines: 250 reads / 5 minutes / subscription; 1,200 writes / hour / subscription. Adaptive throttle manager must initialise from these baselines and observed telemetry, not invented universal constants.

Group contains 3 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-THROTTLE-01: Observe Compute Throttle Budget Headers

Attribute	Value
POC ID	POC-THROTTLE-01
Group	Group 7: API Rate and Throttle Behaviour (FC-16)
Objective	Capture x-ms-ratelimit response headers on Compute calls to calibrate the adaptive throttle manager.
Phase Gate	Phase 1 Pilot
Prerequisites	Pre-Flight complete
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<PROVIDER_SUB>/providers/Microsoft.Compute/capacityReservationGroups?api-version=2024-03-01’ –verbose 2>&1 | tee /tmp/throttle_headers.txt

2. grep -i ‘x-ms-ratelimit’ /tmp/throttle_headers.txt

3. Record remaining budget header names and values

Expected result

Headers visible such as x-ms-ratelimit-remaining-resource and/or x-ms-ratelimit-remaining-subscription-reads. Record values for initial throttle manager configuration.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

POC-THROTTLE-02: Observe 429 Response and Retry-After Header

Attribute	Value
POC ID	POC-THROTTLE-02
Group	Group 7: API Rate and Throttle Behaviour (FC-16)
Objective	Trigger throttling in an isolated environment and capture 429 body and Retry-After.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-THROTTLE-01 Pass. NOTE: Isolated test environment only.
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Submit rapid repeated Compute GET/PUT calls in a controlled loop until 429 is observed

2. Capture full response status, body, and Retry-After header

3. Identify which limit was exhausted (reads, writes, or resource-specific)

4. Stop the loop immediately after evidence capture

Expected result

429 response body and Retry-After seconds recorded; limit type identified.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-THROTTLE-03: Validate Budget Recovery After Throttle

Attribute	Value
POC ID	POC-THROTTLE-03
Group	Group 7: API Rate and Throttle Behaviour (FC-16)
Objective	Confirm that waiting Retry-After allows successful retry and observe remaining budget after recovery.
Phase Gate	Phase 1 Pilot
Prerequisites	POC-THROTTLE-02 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. After 429, wait for the Retry-After period (do not guess a shorter sleep)

2. Re-submit the same request

3. Verify success and capture new rate-limit headers

4. Note any residual throttling

Expected result

Request succeeds after Retry-After when guidance is honored; new remaining budget recorded.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

G7 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

Group 8: Reserved Instance Discount Scope (FC-09)

FinOps validation. RI discounts do not automatically flow from provider to consumer in a shared CRG arrangement. Scope must be validated per provider–consumer pair before any customer cost model is presented (R-44).

Group contains 2 POC entries. Complete result capture for every entry before signing the group roll-up below.

POC-RI-01: Verify RI Discount Scope for Provider Subscription

Attribute	Value
POC ID	POC-RI-01
Group	Group 8: Reserved Instance Discount Scope (FC-09)
Objective	List reservation orders and document applied scopes relative to provider and consumer subscriptions.
Phase Gate	Phase 1 Pilot (FinOps validation before any customer cost modelling)
Prerequisites	FinOps reader access to reservation orders
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. Run:

az reservations reservation-order list –query ‘[].{OrderId:name, DisplayName:displayName}’ -o table

2. For each relevant order:

az reservations reservation-order show –reservation-order-id \<ORDER_ID> -o json

3. Inspect applied scope fields (single subscription, shared, management group, billing account)

4. Determine whether consumer subscription is within applied scope

Expected result

Current scope for each relevant reservation recorded; whether consumer benefits is confirmed or denied; scope type documented.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		
4		

POC-RI-02: Document Discount Gap per Customer Pair

Attribute	Value
POC ID	POC-RI-02
Group	Group 8: Reserved Instance Discount Scope (FC-09)
Objective	For each provider–consumer pair in the pilot set, record whether RI discount flow is confirmed.
Phase Gate	Phase 1 Pilot (required before cost modelling is presented to any customer)
Prerequisites	POC-RI-01 Pass
Executed by	
Execution date	
API version(s) used	
Region / SKU / Zone	

Test steps

1. For each provider–consumer subscription pair, repeat scope inspection from POC-RI-01

2. Fill tracking table: Provider Sub | Consumer Sub | RI Discount Confirmed (Y/N) | Scope Type | Notes

3. Escalate any pair where discount is NOT confirmed before cost models are finalised

Expected result

All pairs have confirmed or denied discount flow documented. Pairs without confirmed discount require scope reconfiguration before customer cost estimates.

Result capture

Field	Entry
Actual Result	
Status	[Not Started] / [Pass] / [Fail] / [Blocked]
Notes / Blocker	
Evidence refs (request IDs, timestamps)	

Step-level observations (optional)

Step #	Observed output / request ID	Pass?
1		
2		
3		

G8 group roll-up sign-off

Check	Result	Initials	Date
All POCs in group executed or formally waived			
Evidence retained (API version, region, SKU, IDs)			
Blockers logged in §7			
Cleanup completed or retention justified			
Group approved to unlock dependents			

4.3 POC numbering reconciliation (workbook ↔ suite ↔ research)

This workbook uses **one** POC-ID scheme — the executable `POC-Test-Scripts/` IDs — so the spec of record and its implementation never diverge. The earlier **research** workbook (`Archive/research/azure_cr_poc_test_workbook.md`) used a broader POC-01…51 exploration scheme; it is retained as historical research, not as a parallel numbering of record.

Asset	Scheme	Status
This workbook (v2.4)	Suite IDs: POC-01…14, POC-30…32, POC-15…20, POC-AKS/VMSS/THROTTLE/RI-, POC-CAP-020/021/001a, POC-PLC-010a, POC-PLC-011(+ELIG/LIVE), POC-CAP-022/024, POC-CAP-023, POC-OPS-006*	**Canonical spec of record**
`POC-Test-Scripts/` (executable)	Same IDs (registry `poc_id`), 49 cases across G1–G12	Implementation of record
`Archive/research/azure_cr_poc_test_workbook.md`	Research POC-01…51 (exploratory)	Historical research; cross-reference only
`acrme_poc_workbook_v2.docx/.pdf` (v2.0)	POC-01…32 subset, binary-only	**Superseded** by this file; archived

Where a v2.4 group case has both an **offline-logic** case and a **-LIVE** case (G9–G12), the offline case validates the deterministic engine logic without Azure, and the `-LIVE` case is the Azure-executed counterpart (BLOCKED until engine/preview APIs are available).

Group 9: Reservation Eligibility & Two-Region Model (CAP-020 / CAP-021 / CAP-001a / PLC-010a)

Validates reservation **eligibility** gating (Availability-Set exclusion and the onboarding placement precondition), the **core-subscription = production** classification, and the **positive** two-region CVAL+DR co-location case that the pre-v2.4 pre-flight wrongly blocked. Implemented in `POC-Test-Scripts/acrme_suite/tests/g9_reservation_eligibility.py`.

> **Normative basis (Baseline v2.4).**
> - **CAP-020 — Availability-Set VMs are ineligible for reservations.** Capacity Reservations cannot be associated with VMs in an Availability Set — the two placement constructs are mutually exclusive. The engine (a) **excludes** AV-Set VMs from eligibility and from the `allocated` / `associated` counts that drive reservation targets (CAP-003); (b) surfaces any managed-scope AV-Set VM as a **non-eligible exception** carrying the CAP-021 remediation action; and (c) never creates/associates/sizes a reservation for such a VM. Eligibility requires **zonal (AZ) placement**, or regional placement only where the SKU lacks zonal reservation support (per the CAP-022 matrix). Maps hard constraint **HC-11 (AVAILABILITY_SET_INELIGIBLE)**.
> - **CAP-021 — Deallocate-or-migrate-to-AZ onboarding precondition.** Before onboarding, a VM must occupy a reservation-eligible placement (AZ preferred; regional only where the SKU has no zonal support). An AV-Set (or otherwise ineligible) VM must first be **deallocated and redeployed into an AZ** (or migrated per the approved runbook). The engine **records the required remediation** and treats the reservation as manageable only once the VM is confirmed eligible; it **does not auto-migrate running workloads** (redeployment is service-impacting and owned by the workload team).
> - **CAP-001a — Core subscription = all production.** VMs in a shared **core** subscription are classified **production** for reservation, buffer, and seed-matrix purposes regardless of any per-workload label, because the core subscription underpins production service. Production buffer (C-2) and production reservation coverage (ENV-001) apply to every managed SKU/AZ in a core subscription. Non-production must not run in the core subscription (ENV-003).
> - **PLC-010a — Two-region CVAL/DR co-location (positive).** In a two-region geography one region hosts Prod and the other hosts **CVAL + DR co-located**; this is a **supported, normative** configuration, not an error. Co-location accounting (HC-6, HC-7, PLC-010) must ensure co-located CVAL is not double-counted as available DR headroom.

Group contains 5 POC entries (4 offline-logic + 1 LIVE). Offline-logic cases run via `python test_v24_reservation_model.py` with no Azure. Complete result capture before signing the group roll-up.

POC-CAP-020: Availability-Set VMs ineligible & uncounted (HC-11)

Attribute	Value
POC ID	POC-CAP-020 (offline-logic)
Requirement	CAP-020 / HC-11
Objective	Assert an Availability-Set VM is **not** reservation-eligible, is **excluded** from `allocated` / `associated` counts, and is surfaced as a non-eligible exception carrying the CAP-021 remediation action.
Phase Gate	Phase 1 Pilot (logic); Azure counterpart POC-CAP-020-LIVE
Prerequisites	none (offline)

Test steps
1. Construct VM descriptors: one in an Availability Set, one zonal (AZ), one regional on a SKU without zonal support.
2. Call `is_reservation_eligible()` on each; call `counts_toward_reservation_target()` on each; call `required_remediation()` on the AV-Set VM.

Expected result
- AV-Set VM → `is_reservation_eligible == False`; `counts_toward_reservation_target == False`; `required_remediation` == the CAP-021 deallocate/redeploy-to-AZ action.
- Zonal VM → eligible and counted. Regional VM on a non-zonal SKU → eligible (regional placement) per CAP-022 matrix.

Result capture

Field	Value
Executed by / date	
Eligibility results (AV-Set / zonal / regional)	
Remediation string recorded	
Status (Pass/Fail/Blocked)	

POC-CAP-021: Deallocate/redeploy-to-AZ onboarding precondition

Attribute	Value
POC ID	POC-CAP-021 (offline-logic)
Requirement	CAP-021
Objective	Assert an ineligible VM is treated as **not manageable** until confirmed in an eligible placement, and that the engine records the remediation without auto-migrating.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps
1. For an AV-Set VM, call `is_manageable()` (expect False) and `required_remediation()`.
2. Redeploy (simulate) into an AZ; re-evaluate `is_manageable()` (expect True).

Expected result — Manageable flips False → True only after eligible placement; remediation action recorded on the ineligible state; no auto-migration is performed by the engine.

Result capture

Field	Value
Executed by / date	
Manageable before / after	
Status (Pass/Fail/Blocked)	

POC-PLC-010a: Two-region CVAL/DR co-location is valid (positive)

Attribute	Value
POC ID	POC-PLC-010a (offline-logic)
Requirement	PLC-010a / ENV-003 / REG-003
Objective	Assert a two-region geography config (Prod in region A; CVAL+DR co-located in region B) **passes** validation — the positive case the pre-v2.4 PF-09/PF-10 wrongly blocked.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline; builds an in-memory two-region `Config`)

Test steps
1. Build a two-region `Config` (Prod=region A, NonProd/CVAL=region B, DR=region B) and run `_populate()` / `validate()`.
2. Assert validation succeeds and DR==NonProd region is **not** flagged as a violation.
3. Negative control: a single-region config (Prod==NonProd) must still fail.

Expected result — Two-region co-location config validates cleanly; Prod≠NonProd enforced; DR==NonProd accepted under `distribution_model = two-region`. Middle East geography remains `DR_NOT_OFFERED` (DEC-001) and is asserted separately as not-a-general-outcome.

Result capture

Field	Value
Executed by / date	
Two-region validation result	
Negative control result	
Status (Pass/Fail/Blocked)	

POC-CAP-001a: Core subscription classified all-production

Attribute	Value
POC ID	POC-CAP-001a (offline-logic)
Requirement	CAP-001a
Objective	Assert every VM in a core subscription is classified **production** for buffer/reservation/seed-matrix regardless of per-workload label.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps

1. Classify VMs in a core subscription carrying mixed workload labels via `classify_for_buffer()`.
2. Assert all resolve to production buffer policy (C-2) and production reservation coverage (ENV-001).

Expected result — All core-subscription VMs classified production irrespective of label; non-production placement in the core subscription is rejected (ENV-003).

Result capture

Field	Value
Executed by / date	
Classification results	
Status (Pass/Fail/Blocked)	

POC-CAP-020-LIVE: Azure rejects AV-Set VM reservation association

Attribute	Value
POC ID	POC-CAP-020-LIVE (Azure-executed)
Requirement	CAP-020 / HC-11
Objective	Confirm Azure itself rejects associating an Availability-Set VM with a Capacity Reservation, corroborating the offline eligibility gate.
Phase Gate	Phase 2 (requires engine / live Azure)
Prerequisites	Provider CRG + reservation (POC-01/02); an AV-Set test VM

Test steps
1. Create a VM in an Availability Set.
2. Attempt to associate it with a Capacity Reservation (`az vm update --capacity-reservation-group …`).

Expected result — Azure returns an error (AV-Set/CR mutual exclusivity); the engine surfaces it as a CAP-020 non-eligible exception with CAP-021 remediation.

Status: BLOCKED until engine / live Azure available (returns `blocked` , consistent with POC-18/19/20).

Result capture

Field	Value
Executed by / date	
Azure error code/message	
Status (Pass/Fail/Blocked)	

G9 group roll-up sign-off

Field	Value
All G9 offline-logic cases Pass? (POC-CAP-020/021/001a, POC-PLC-010a)	
LIVE case status (POC-CAP-020-LIVE)	
Reviewer / date	

Group 10: Even Zone Distribution & Rebalance (PLC-011, A.9, C-13)

Validates the **even ≈1/zone_count** zone-distribution target, drift/tolerance arithmetic, greatest-deficit placement preference, and the rebalancing trigger. Reproduces the Calculation-Logic Reference **Scenario 21** worked example exactly. Implemented in `POC-Test-Scripts/acrme_suite/tests/g10_zone_distribution.py` .

> **Normative basis (Baseline v2.4).**
> - **PLC-011 — Even zone-distribution target & rebalancing.** Placement targets an even spread of a workload's VMs across a region's AZs — approximately `1 / zone_count` per zone (≈33% each in a three-zone region). This is a placement **target**, not merely the PLC-007 ε zone-diversity scoring signal. When a new deployment or growth event would skew a workload's zone distribution beyond a **configurable tolerance** from even, the engine must (a) **prefer the under-represented zone(s)** for new placement, and (b) raise a **rebalancing recommendation/action** (subject to approval and Azure feasibility). Target ratio and tolerance are configuration-driven (`PlacementPolicy`).
> - **C-13 — Zone-distribution target & tolerance** (configurable): even ≈ `1/zone_count` per zone with a configurable skew tolerance before rebalancing.
> - **Appendix A.9 (count/skew variant).** `Target VMs per zone = round(total_vms / zone_count)` ; `Skew(z) = count(z) − target` ; `Max Skew = max_z |Skew(z)|` ; `Rebalance Trigger = Max Skew > Configured Skew Tolerance (C-13)` ; `Preferred Placement Zone = argmin_z count(z)` (fill the most under-represented zone first).

Group contains 3 POC entries (2 offline-logic + 1 LIVE).

POC-PLC-011: Even distribution + rebalance — reproduce Scenario 21

Attribute	Value
POC ID	POC-PLC-011 (offline-logic)
Requirement	PLC-011 / C-13 / A.9
Objective	Reproduce Calc-Logic **Scenario 21** exactly and assert drift, rebalance trigger, chosen zone, and the A.9 count-variant skew.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Scenario 21 worked example (share variant, `tolerance = 0.10`)

Zone	VMs	Share	Target share	Deficit (target − share)
az1	14	0.467	0.3333	−0.133
az2	9	0.300	0.3333	+0.033
az3	7	0.233	0.3333	+0.100
Total	**30**	1.000	—	—

- `max_drift = 0.134` (az1, |0.467 − 0.3333|) **> 0.10 tolerance ⇒ rebalance = TRUE**.
- `chosen_zone = argmax(deficit) = az3` (most under-represented).
- After rebalancing to 10/10/10 → every share = 0.3333, drift = 0.

A.9 count variant — `Target VMs/zone = round(30/3) = 10`; Skew az1/az2/az3 = **+4 / −1 / −3**; `Max Skew = 4`; `Preferred Placement Zone = argmin(count) = az3`.

Test steps — Call `zone_target_share`, `zone_shares`, `zone_deficits`, `max_drift`, `rebalance_needed`, `chosen_zone`, then the A.9 helpers `target_vms_per_zone`, `zone_skew`, `max_skew`, `preferred_placement_zone` on the 14/9/7 input.

Expected result — All values match the table above exactly (target_share 0.3333; deficits −0.133/+0.033/+0.100; max_drift 0.134; rebalance TRUE; chosen_zone az3; skew +4/−1/−3; max_skew 4; preferred zone az3).

Result capture

Field	Value
Executed by / date	
max_drift / rebalance / chosen_zone	
A.9 max_skew / preferred zone	
Status (Pass/Fail/Blocked)	

POC-PLC-011-ELIG: Greatest-deficit eligibility fallback

Attribute	Value
POC ID	POC-PLC-011-ELIG (offline-logic)
Requirement	PLC-011
Objective	Assert that when the greatest-deficit zone is ineligible, placement falls back to the next-greatest-deficit **eligible** zone.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps — With the Scenario-21 deficits, mark `az3` ineligible and call `preferred_placement_zone(eligible=...)`.

Expected result — Chosen zone becomes the next-greatest-deficit eligible zone (az2); no placement into an over-represented zone.

Result capture

Field	Value
Executed by / date	
Chosen eligible zone	
Status (Pass/Fail/Blocked)	

POC-PLC-011-LIVE: Engine steers placement & raises rebalance action

Attribute	Value
POC ID	POC-PLC-011-LIVE (Azure/engine-executed)
Requirement	PLC-011
Objective	Confirm the live engine places new VMs toward the even target and raises a rebalancing action when skew exceeds tolerance.
Phase Gate	Phase 2 (requires engine / preview APIs)
Prerequisites	Engine reconciliation loop; a region with ≥3 AZs

Expected result — New placements prefer the under-represented zone; a skew beyond C-13 tolerance produces a rebalancing recommendation/action.

Status: BLOCKED until engine/preview APIs available.

Result capture

Field	Value
Executed by / date	
Placement/rebalance observed	
Status (Pass/Fail/Blocked)	

G10 group roll-up sign-off

Field	Value
POC-PLC-011 reproduces Scenario 21 exactly?	
POC-PLC-011-ELIG Pass?	
LIVE case status	
Reviewer / date	

Group 11: Seed Matrix & Reactive Discovery (CAP-022 / CAP-024)

Validates the **seed-at-0** SKU/AZ matrix generation under **product-team budget governance**, and the **reactive discovery** auto-create + scope-file governance reconciliation. Implemented in `POC-Test-Scripts/acrme_suite/tests/g11_seed_matrix.py`.

> **Normative basis (Baseline v2.4).**
> - **CAP-022 — Seed-at-0 eligible-SKU/AZ matrix & budget governance** (extends CAP-009). The managed set of SKU/VM-family × region × AZ combinations is initialised as a **seed matrix of count-0 reservations** (“seed reservations”): every eligible combination in the scope file is created in Azure at reserved quantity **0** inside the correct regional or per-AZ CRG (CAP-023), so reconciliation can scale each up from a known baseline the instant allocated demand appears (CAP-007). A combination enters the matrix **only** with a named owning **product team** and an approved **budget line** (any reservation scaled above 0 incurs cost). The matrix, eligibility rules, and budget owners are configuration-driven (scope file, CAP-019) and versioned. (A “seed reservation” is a zero-count reservation — not the PLC-003 placement seed record.)
> - **CAP-024 — Reactive SKU/AZ discovery reconciled with governance** (reconciles CAP-019). When reconciliation/deployment detects an **allocated VM of a SKU/AZ not yet in the seed matrix**, the engine **auto-creates** the reservation — CRG (if absent) + reservation sized at `allocated + buffer` (CAP-003) in the correct per-AZ/regional CRG (CAP-023) — so production is protected immediately, **and simultaneously raises a scope-file governance item** (CAP-019) so the discovered SKU/AZ is ratified with an owning team and budget (CAP-022). Auto-creation protects capacity first; governance then makes it authoritative (or triggers approved decommissioning rollback, CAP-010, if rejected). Preserves CAP-002 (Azure resource precedes config activation); scope file reconciled **within the governance SLA**, not as a precondition of protecting live production.

Group contains 3 POC entries (2 offline-logic + 1 LIVE).

POC-CAP-022: Seed-at-0 SKU/AZ matrix + product-team budget gate

Attribute	Value
POC ID	POC-CAP-022 (offline-logic)
Requirement	CAP-022 (extends CAP-009)
Objective	Generate a count-0 seed matrix across eligible SKU×region×AZ and enforce that only combinations with a named product team + approved budget line are admitted.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps

1. `build_seed_matrix()` over a scope-file fixture (eligible SKU/AZ set) → assert every entry is `count == 0` and mapped to the correct regional/per-AZ CRG (CAP-023).
2. `budget_gate()` / `approved_seed_matrix()` → assert combinations lacking an owning team or approved budget are **excluded** as governance exceptions.

Expected result — All admitted seed reservations are count-0 and CRG-correct; unbudgeted/unowned combinations are rejected; matrix is deterministic and versionable.

Result capture

Field	Value
Executed by / date	
Matrix size / all count-0?	
Rejected (unbudgeted) entries	
Status (Pass/Fail/Blocked)	

POC-CAP-024: Reactive SKU/AZ discovery auto-create + governance item

Attribute	Value
POC ID	POC-CAP-024 (offline-logic)
Requirement	CAP-024 (reconciles CAP-019)
Objective	On discovery of an allocated SKU/AZ not in the matrix, assert auto-create (sized `allocated + buffer`, correct CRG) **and** a raised scope-file governance item.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps — Feed `reactive_discovery()` an allocated SKU/AZ absent from the seed matrix; inspect the returned action set.

Expected result — Returns (a) an auto-create action: CRG-if-absent + reservation sized `allocated + buffer` (CAP-003) in the correct per-AZ/regional CRG; **and** (b) a scope-file governance item (CAP-019) naming the SKU/AZ for ratification with owning team + budget (CAP-022). Production protected first; governance reconciled within SLA.

Result capture

Field	Value
Executed by / date	
Auto-create action (size/CRG)	
Governance item raised?	
Status (Pass/Fail/Blocked)	

POC-CAP-024-LIVE: Azure auto-create + scope-file governance item

Attribute	Value
POC ID	POC-CAP-024-LIVE (Azure/engine-executed)
Requirement	CAP-024
Objective	Confirm the live engine auto-creates the reservation in Azure and files the scope-file governance item on reactive discovery.
Phase Gate	Phase 2 (requires engine / live Azure)
Prerequisites	Engine reconciliation loop; scope file; deploy an unmanaged SKU/AZ

Expected result — Azure reservation auto-created (sized allocated+buffer, correct CRG); governance item raised for ratification.

Status: BLOCKED until engine / live Azure available.

Result capture

Field	Value
Executed by / date	
Azure resource created	
Governance item ref	
Status (Pass/Fail/Blocked)	

G11 group roll-up sign-off

Field	Value
POC-CAP-022 / POC-CAP-024 offline Pass?	
LIVE case status	
Reviewer / date	

Group 12: Regional + Per-AZ CRG & Deterministic Naming (CAP-023, OPS-006 / C-12)

Validates the per-environment **regional + per-AZ CRG structure** and the **deterministic naming convention + counter** (RG/CRG/subscription). Implemented in `POC-Test-Scripts/acrme_suite/tests/g12_naming_convention.py`.

> **Normative basis (Baseline v2.4).**
> - **CAP-023 — Regional and per-AZ CRG structure** (extends CAP-011). For each **environment** (Prod, CVAL/NonProd, DR) within a subscription and region, reservations are organised as **one regional (non-zonal) CRG plus one CRG per availability zone**. Example (Prod in East US 2, 3 zones): `crg-pr-eus2-reg` (regional, SKUs without zonal support), `crg-pr-eus2-az1`, `crg-pr-eus2-az2`, `crg-pr-eus2-az3`. Per-AZ CRGs hold zone-bound reservations and enforce CAP-011 zone isolation **structurally** (zone-1 capacity can never be counted as available in another zone); the regional CRG holds only non-zonal-support SKUs. Environments are **never** mixed within a CRG (ENV-003). Names follow OPS-006 / C-12.
> - **OPS-006 / C-12 — Naming convention & counter.** All managed **RGs, CRGs, and subscriptions** follow a deterministic, parseable convention carrying environment, geography/region, purpose/scope tokens, and a zero-padded instance **counter**. Reference patterns: **RG** `rg-odcr-<env>-<region>-<NN>` (e.g. `rg-odcr-prod-eus2-01`); **CRG** `crg-<env>-<region>-<scope>` where `scope ∈ {reg, az1, az2, az3}` (e.g. `crg-pr-eus2-reg`, `crg-pr-eus2-az1/az2/az3`); **subscription** `sub-<org>-<domain>-<purpose>-<NN>` (e.g. `sub-jda-cld-core-01`). Convention, tokens, and counter width are configuration-driven (C-12); the engine validates managed resources and flags non-conforming names as governance exceptions. CRG scope tokens map directly to the CAP-023 structure.

Group contains 3 POC entries (2 offline-logic + 1 LIVE).

POC-CAP-023: Regional + per-AZ CRG structure per environment

Attribute	Value
POC ID	POC-CAP-023 (offline-logic)
Requirement	CAP-023 (extends CAP-011)
Objective	Assert that for each environment/region the expected CRG set is exactly one regional CRG + one CRG per AZ, with no environment mixing.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps

1. `expected_crg_set(env, region, zone_count)` for Prod/East US 2/3 zones → assert `{crg-pr-eus2-reg, crg-pr-eus2-az1, crg-pr-eus2-az2, crg-pr-eus2-az3}`.
2. Assert environments are never mixed within a CRG (ENV-003); per-AZ CRGs are zone-bound.

Expected result — Exact regional + per-AZ CRG set per environment; structure enforces zone isolation; scope tokens match OPS-006.

Result capture

Field	Value
Executed by / date	
Expected CRG set produced	
Status (Pass/Fail/Blocked)	

POC-OPS-006: Deterministic RG/CRG/subscription naming + counter

Attribute	Value
POC ID	POC-OPS-006 (offline-logic)
Requirement	OPS-006 / C-12
Objective	Validate conforming names pass and non-conforming names are flagged as governance exceptions, across RG/CRG/subscription patterns and the zero-padded counter.
Phase Gate	Phase 1 Pilot (logic)
Prerequisites	none (offline)

Test steps
1. `validate_rg_name`, `validate_crg_name`, `validate_subscription_name` on conforming samples (`rg-odcr-prod-eus2-01`, `crg-pr-eus2-az1`, `sub-jda-cld-core-01`) → expect pass.
2. Same on malformed samples (bad scope token, missing/short counter, wrong env token) → expect flagged; run `flag_nonconforming()` over a mixed set.

Expected result — Conforming names validate; malformed names (bad CRG scope, short counter, wrong env) are flagged as governance exceptions.

Result capture

Field	Value
Executed by / date	
Conforming pass / malformed flagged	
Status (Pass/Fail/Blocked)	

POC-CAP-023-LIVE: Enumerate real CRGs & assert structure/ naming

Attribute	Value
POC ID	POC-CAP-023-LIVE (Azure-executed)
Requirement	CAP-023 / OPS-006
Objective	Enumerate real CRGs in a managed subscription/region and assert the regional+per-AZ structure and naming convention hold.
Phase Gate	Phase 2 (requires engine / live Azure)
Prerequisites	Managed subscription with provisioned CRGs

Expected result — Live CRGs match the CAP-023 structure and OPS-006 naming; non-conforming resources flagged.

Status: BLOCKED until engine / live Azure available.

Result capture

Field	Value
Executed by / date	
CRGs enumerated / conforming	
Status (Pass/Fail/Blocked)	

G12 group roll-up sign-off

Field	Value
POC-CAP-023 / POC-OPS-006 offline Pass?	
LIVE case status	
Reviewer / date	

---

5. Test Progress Tracker

Master tracker for programme status reporting. Update Status as tests complete. All Status cells start as [Not Started].

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Completed	Notes
POC-01	Create CRG in Provider Subscription	G1	Phase 1		[Not Started]		
POC-02	Create Capacity Reservation within the CRG	G1	Phase 1		[Not Started]		
POC-03	Associate VM with Reservation (Provider)	G1	Phase 1		[Not Started]		
POC-04	Verify Capacity Consumption Count Increments	G1	Phase 1		[Not Started]		
POC-05	Disassociate VM — Verify Capacity Released	G1	Phase 1		[Not Started]		
POC-06	Enable Cross-Subscription Sharing on CRG	G2	Phase 1		[Not Started]		
POC-06a	Cross-Subscription Zone Align-	G2	Phase 1 Gate		[Not Started]		

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Com-pleted	Notes
	ment (FC-06)						
POC-07	Con-sumer Discov-ers Shared CRG via ARG	G2	Phase 1		[Not Started]		
POC-08	Associate Con-sumer VM with Shared CRG	G2	Phase 1		[Not Started]		
POC-09	Verify Com-bined Con-sumption Count	G2	Phase 1		[Not Started]		
POC-10	100-Con-sumer Limit Boundary Beha-viour	G2	Produc-tion		[Not Started]		
POC-11	Create DR Re-gion CRG and Re-servation	G3	Phase 1		[Not Started]		
POC-12	Simulate DR Fail-over	G3	Phase 2		[Not Started]		
POC-13	DR Floor Protec-	G3	Phase 1		[Not Started]		

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Completed	Notes
	tion Verification						
POC-14	Failback — Restore Primary, Release DR	G3	Phase 2		[Not Started]		
POC-15	Zero-Quantity Reservation Behaviour	G5	Phase 1		[Not Started]		
POC-16	ARG Indexing Delay Measurement	G5	Phase 1		[Not Started]		
POC-17	Concurrent Association Safety	G5	Phase 1		[Not Started]		
POC-18	Tier 1 Automatic Capacity Increase	G5	Phase 2		[Not Started]		
POC-19	Tier 2 Approval Gate	G5	Phase 2		[Not Started]		
POC-20	Tier 3 Rejection in Phase 1	G5	Phase 1		[Not Started]		
POC-30	Verify Quota Group	G4	Phase 1 Gate		[Not Started]		

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Completed	Notes
	API Availability						
POC-31	Add Subscription to Quota Group and Verify Headroom	G4	Phase 2		[Not Started]		
POC-32	Release and Reuse Behaviour	G4	Phase 2		[Not Started]		
POC-AKS-01	AKS Node Pool CRG Association at Creation	G6	Phase 2		[Not Started]		
POC-AKS-02	Existing Node Pool CRG Change Requires Recreation (FC-18)	G6	Phase 1		[Not Started]		
POC-VMSS-01	VMSS Uniform CRG Association	G6	Phase 2		[Not Started]		
POC-VMSS-02	VMSS Flexible CRG Association	G6	Phase 2		[Not Started]		
	VMSS Disasso-	G6	Phase 2				

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Com-pleted	Notes
POC-VMSS-03	ciation Beha-viour				[Not Started]		
POC-VMSS-DR	VMSS Zone Outage via Shared CRG (FC-08)	G6	Phase 2 Gate		[Not Started]		
POC-THROTTLE-01	Observe Compute Throttle Budget Headers	G7	Phase 1		[Not Started]		
POC-THROTTLE-02	Observe 429 Re-sponse and Retry-After	G7	Phase 1		[Not Started]		
POC-THROTTLE-03	Validate Budget Recovery After Throttle	G7	Phase 1		[Not Started]		
POC-RI-01	Verify RI Discount Scope for Provider Subscrip-tion	G8	Phase 1		[Not Started]		
POC-RI-02	Docu-ment Discount Gap per Custom-er Pair	G8	Phase 1		[Not Started]		

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Completed	Notes
POC-CAP-020	AV-Set VMs ineligible & uncounted (HC-11)	G9	Phase 1 (logic)		[Not Started]		
POC-CAP-021	Deallocate/redeploy-to-AZ onboarding precondition	G9	Phase 1 (logic)		[Not Started]		
POC-PLC-010a	Two-region CVAL/DR co-location valid (positive)	G9	Phase 1 (logic)		[Not Started]		
POC-CAP-001a	Core subscription classified all-production	G9	Phase 1 (logic)		[Not Started]		
POC-CAP-020-LIVE	Azure rejects AV-Set VM reservation association	G9	Phase 2 (live)		[Blocked]		Engine/live Azure required
POC-PLC-011	Even distribution + rebalance (Scenario 21)	G10	Phase 1 (logic)		[Not Started]		
		G10					

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Com-pleted	Notes
POC-PLC-011-ELIG	Greatest-deficit eligibility fallback		Phase 1 (logic)		[Not Started]		
POC-PLC-011-LIVE	Engine steers place-ment & raises re-balance action	G10	Phase 2 (live)		[Blocked]		Engine/ preview APIs re-quired
POC-CAP-022	Seed-at-0 SKU/ AZ mat-rix + budget gate	G11	Phase 1 (logic)		[Not Started]		
POC-CAP-024	Reactive SKU/AZ discovery auto-create + gov-ernance item	G11	Phase 1 (logic)		[Not Started]		
POC-CAP-024-LIVE	Azure auto-create + scope-file gov-ernance item	G11	Phase 2 (live)		[Blocked]		Engine/ live Azure re-quired
POC-CAP-023	Regional + per-AZ CRG structure per en-viron-ment	G12	Phase 1 (logic)		[Not Started]		

POC ID	Test Name	Group	Phase Gate	Owner	Status	Date Completed	Notes
POC-OPS-006	Deterministic RG/CRG/subscription naming + counter	G12	Phase 1 (logic)		[Not Started]		
POC-CAP-023-LIVE	Enumerate real CRGs & assert structure/naming	G12	Phase 2 (live)		[Blocked]		Engine/live Azure required

6. Phase Gate Sign-Off Checklist

Phase gates are cumulative. Phase 2 cannot open until Phase 1 is fully signed. Production cannot open until Phase 2 is fully signed and residual Critical/High risks are accepted.

6.1 Phase 1 (Pilot) Gate

All items must be Pass before pilot customers are onboarded. Scope: discovery, inventory, zone-resolution validation, placement recommendations, quota visibility, sharing setup in approved subscriptions, CR quantity increases, and tested Tier 1 operations only.

Gate Item	Required POC / Evidence	Status	Sign-off Authority	Sign-off Date
POC-01 to POC-09 foundational sharing path pass	POC-01 … POC-09	[Not Started]	Test Lead + Capacity Architect	
POC-06a zone alignment confirmed for every provider–consumer pair	POC-06a	[Not Started]	Placement Owner	
POC-30 quota group API available in every pilot scope (or formal fallback approved)	POC-30	[Not Started]	Quota Owner	
POC-15 and POC-16 safety behaviours documented	POC-15, POC-16	[Not Started]	Compute Owner + Engineering	
POC-THROTTLE-01 baselines recorded for target subscriptions	POC-THROTTLE-01	[Not Started]	SRE	
G-14 permissions model security reviewed (Tier 3 remains disabled if unresolved)	POC-20 + Security review	[Not Started]	Security	
POC-RI-01 discount scope validated before customer cost modelling	POC-RI-01	[Not Started]	FinOps	
Geography-aware region model confirmed	PF-09, PF-10, POC-PLC-010a	[Not Started]	DR Architect	

Gate Item	Required POC / Evidence	Status	Sign-off Authority	Sign-off Date
per `distribution_model` (three-region distinct; two-region CVAL/DR co-location; Middle East `DR_NOT_OFFERED`)				
v2.4 reservation-model offline logic passes (G9–G12: CAP-020/021/001a, PLC-010a/011, CAP-022/023/024, OPS-006)	`python test_v24_reservation_model.py` + POC-CAP-020/021/001a, POC-PLC-010a/011, POC-CAP-022/023/024, POC-OPS-006	[Not Started]	Capacity Architect	
POC-20 Tier 3 rejection confirmed in Phase 1 mode	POC-20	[Not Started]	DR Owner + Security	
Preview risk formally accepted for bounded pilot	PG-10 governance	[Not Started]	Product Owner / Board	

6.2 Phase 2 (Controlled Automation) Gate

All Phase 1 gates plus the following:

Gate Item	Required POC / Evidence	Status	Sign-off Authority	Sign-off Date
All Phase 1 gates closed	Phase 1 checklist	[Not Started]	Test Lead	
POC-31 and POC-32 pass with measured propagation	POC-31, POC-32	[Not Started]	Quota Owner	
POC-11 to POC-14 DR failover and failback validated	POC-11 … POC-14	[Not Started]	DR Owner	
POC-18 and POC-19 automation and approval gate verified	POC-18, POC-19	[Not Started]	Engineering + DR Owner	
POC-VMSS-DR documented (pass or fail both acceptable)	POC-VMSS-DR	[Not Started]	Compute Owner	
POC-RI-02 discount gap documented for all pilot customers	POC-RI-02	[Not Started]	FinOps	
All Phase 1 blockers resolved or explicitly accepted	Section 7 log	[Not Started]	Programme Manager	
G-15 engine mode state machine implemented and fault-injection tested	Engine mode tests	[Not Started]	Engineering	

6.3 Production Gate

Unrestricted autonomous DR, destructive capacity transfer, and VMSS emergency automation remain prohibited until these gates close.

Gate Item	Required POC / Evidence	Status	Sign-off Authority	Sign-off Date
All Phase 2 gates closed	Phase 2 checklist	[Not Started]	Programme Manager	
POC-10 consumer limit documented and sharding strategy approved	POC-10	[Not Started]	Capacity Architect	
POC-20 Tier 3 controls confirmed; Tier 3 only if G-14 closed and separately authorized	POC-20 + G-14	[Not Started]	Security + Board	
POC-THROTTLE-02 and POC-THROTTLE-03 recorded for all target regions	POC-THROTTLE-02/03	[Not Started]	SRE	
Full DR failover and failback exercised with pilot customers	PG-07	[Not Started]	DR Owner + Business Continuity	
All Critical and High residual risks accepted by leadership	Risk register R-01…R-44	[Not Started]	Board / Design Review Council	
PG-01 through PG-10 production entry gates evidenced	Architecture §17	[Not Started]	Design Review Council	

6.4 Production entry gates (architecture PG-01 … PG-10)

Gate	Required evidence	Exit criterion	Status
PG-01 Sharing	Executed sharing, unsharing, discovery, 100-consumer tests	Stable behaviour + governance acceptance	[Not Started]
PG-02 Quota Groups	Executed POC-30 to POC-32 (+ POC-33 if in scope)	API, eligibility, membership, propagation documented	[Not Started]
PG-03 CR updates	Increase, floor, zero, running-VM tests	Scenario matrix approved	[Not Started]
PG-04 State machine	Formal model + fault-injection (G-15)	No illegal steady-state/DR transitions	[Not Started]
PG-05 Credentials	Approved UAMI or alternative (G-14)	Least privilege, consent, rotation, revocation tested	[Not Started]
PG-06 Placement	Concurrency and replay tests	No over-allocation under parallel requests	[Not Started]
PG-07 DR	Full failover and failback exercise	Application and data recovery objectives demonstrated	[Not Started]
PG-08 VMSS	Uniform and Flexible matrix + POC-VMSS-DR	Phase scope explicitly enforced	[Not Started]
PG-09 Scale	100 / 500 / 1,000 / multi-thousand scenarios	Adaptive reconciliation meets internal SLO	[Not Started]
PG-10 Preview	Architecture governance decision	Preview risk accepted or dependency replaced	[Not Started]

6.5 Residual risk acceptance (production entry)

Even after controls, residual platform and operational risks remain (architecture §15, §40). Leadership must explicitly accept or mitigate the following before Production gate closure. This table is not a substitute for the full risk register.

Risk ID	Summary	Residual level	Acceptance owner	Accepted Y/N	Date
R-01	CRG sharing preview changes or withdrawn	High	Product Owner		
R-05	Quota Groups unavailable in target scope	High	Quota Owner		
R-08	Quota release propagation delayed	Medium	DR Owner		
R-12	Non-paired regions correlated dependencies	Medium	DR Architect		
R-18	Tier 3 modifies wrong consumer VM	High if enabled	Security		
R-19	Credential model grants excessive rights	Medium	Security		
R-23	Concurrent placements over-assign	Low after holds	Engineering		
R-25	ARM throttling delays DR actions	Medium	SRE		
R-41	Cross-sub zone mapping mismatch	Medium after POC-06a	Engineering		
R-42	VMSS shared CRG zone-	High until GA/fallback	Compute Owner		

Risk ID	Summary	Residual level	Acceptance owner	Accepted Y/N	Date
	outage repro-vision				
R-44	RI discounts do not flow to consumer	Low after POC-RI	FinOps		

Unacceptable without separate board authorization: autonomous Tier 3 VM disassociation; any automated Tier 3 VMSS operation; DR activation without authoritative engine state machine; treating preview or POC behaviour as a Microsoft commitment; using stale ARG data for destructive actions.

7. Issue and Blocker Log

Log every Blocked or Fail outcome that stops a gate. Known programme blockers at workbook issue: G-14 (consumer credential model), G-15 (engine mode state machine), B-1 (quota group availability).

Blocker ID	POC ID	Date Raised	Description	Impact	Assigned To	Status	Resolution Date	Resolution Notes
B-G14	POC-20 / Tier 3		Consumer credential model unresolved (UAMI vs SP)	Tier 3 blocked; production blocker	Security	[Open]		
B-G15	POC-18 / mode		EngineModeState machine incomplete	DR automation blocked	Engineering	[Open]		
B-B1	POC-30		Quota Groups API availability unproven in target scopes	Quota-group engineering blocked until pass	Quota Owner	[Open]		
B-FC08	POC-VMSS-DR		VMSS reprovision via shared CRG during zone outage is Preview limitation	Phase 2 VMSS DR design gated	Compute Owner	[Open]		
						[Open]		
						[Open]		

Block-er ID	POC ID	Date Raised	De-scrip-tion	Impact	As-signed To	Status	Resol-ution Date	Resol-ution Notes
						[Open]		
						[Open]		
						[Open]		
						[Open]		
						[Open]		
						[Open]		

8. Assumptions Being Validated

Source: Production-Readiness Review §8 Assumption Validation Matrix (A-01 through A-20). Update Result after the validating POC(s) execute. Board disposition remains with the Design Review Council.

Assumption ID	Assumption (plain language)	Current Status	Validating POC(s)	Result
A-01	Shared CRG is suitable for production dependency	Unproven preview dependency	POC-06 … POC-10, PG-01	
A-02	100-consumer limit per CRG is the operative limit	Validation required	POC-10	
A-03	Consumer can discover shared CRGs through normal list operations	Challenged — use ARG + provider inventory	POC-07	
A-04	Two quota groups can be created in every target scope and region	Unproven — hard gate	POC-30	
A-05	Quota-group membership removes subscription-level quota checks	Rejected — dual validation mandatory	POC-31	
A-06	Azure natively protects the DR share within NonProd plus DR	Rejected — engine floor only	POC-13	
A-07	NonProd reduction immediately funds DR expansion	Unproven — measure propagation	POC-31, POC-32, POC-19	
A-08	Quantity can safely reduce to zero while VMs run	Unproven as general guarantee	POC-15	

Assumption ID	Assumption (plain language)	Current Status	Validating POC(s)	Result
A-09	30–40% DR baseline is sufficient	Unsupported business assumption	POC-11, POC-13 + workload analysis	
A-10	30% emergency headroom is economically and operationally optimal	Unsupported — tunable parameter	POC-18 + scenario tests	
A-11	NonProd and DR co-location is acceptable	Conditional — customer approval required	PF-09/10, POC-11	
A-12	Sequential placement is near-optimal	Unproven — shadow joint optimization	Placement shadow tests (engine)	
A-13	Five-minute reconciliation is sufficient	Unproven at scale	Scale / churn tests	
A-14	ARG is fresh enough for operational decisions	Rejected for destructive actions	POC-16	
A-15	Auto-increase is safe	Conditional — approval-gated in Phase 1	POC-18	
A-16	Tier 1 can always be automated	Challenged — requires mode gate and fresh checks	POC-18	
A-17	Tier 2 is non-destructive	Partly true — changes NonProd guarantees	POC-19	
A-18	Tier 3 can be implemented with existing credentials	Rejected — G-14 blocker	POC-20	

Assumption ID	Assumption (plain language)	Current Status	Validating POC(s)	Result
A-19	VMSS can follow single-VM Path B	Rejected — Phase 1 limitation	POC-VMSS-01…03, POC-15	
A-20	Cosmos throughput can be predetermined from entity counts	Rejected — size from measured workload	Load tests (out of this workbook)	

Appendix A — Azure CLI Quick Reference

One-page operational cheat sheet. Prefer the exact commands embedded in each POC when versions differ.

Login and subscription

az login

az account set –subscription \<SUBSCRIPTION_ID>

az account show

az account show –query id -o tsv

Resource provider check

az provider show –namespace Microsoft.Compute –query registrationState -o tsv

az provider show –namespace Microsoft.Quota –query registrationState -o tsv

az provider register –namespace Microsoft.Compute

az provider register –namespace Microsoft.Quota

CRG commands

az capacity reservation group list –resource-group \<RG> -o table

az capacity reservation group show –resource-group \<RG> –name \<CRG_NAME> -o json

az capacity reservation group create –resource-group \<RG> –name \<CRG_NAME> –location \<REGION> –zones 1 2 3

az capacity reservation group delete –resource-group \<RG> –name \<CRG_NAME> –yes

Capacity Reservation commands

az capacity reservation list –resource-group \<RG> –capacity-reservation-group \<CRG_NAME> -o table

az capacity reservation show –resource-group \<RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> -o json

az capacity reservation create –resource-group \<RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –sku \<VM_SKU> –capacity \<N> –location \<REGION>

az capacity reservation update –resource-group \<RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –capacity \<N>

az capacity reservation delete –resource-group \<RG> –capacity-reservation-group \<CRG_NAME> –name \<CR_NAME> –yes

VM associate / disassociate

az vm create ... –capacity-reservation-group \<CRG_RESOURCE_ID>

az vm show –resource-group \<RG> –name \<VM> –query capacityReservation -o json

az vm deallocate –resource-group \<RG> –name \<VM>

az vm update –resource-group \<RG> –name \<VM> –set capacityReservation.capacityReservationGroup=null

Sharing profile (REST)

az rest –method PATCH –url ‘https://management.azure.com/subscriptions/\<SUB>/resourceGroups/\<RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG>?api-version=2024-03-01’ –body ‘{“properties”:{“sharingProfile”:{“subscriptionIds”:[{“id”:”/subscriptions/\<CONSUMER_SUB>“}]}}}’

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<SUB>/resourceGroups/\<RG>/providers/Microsoft.Compute/capacityReservationGroups/\<CRG>?api-version=2024-03-01’ –query properties.sharingProfile

Zone mapping

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<SUB>/locations?api-version=2022-12-01’ –query “[?name==’\<REGION>‘].availabilityZoneMappings”

ARG query

az graph query -q “Resources | where type =\~ ‘microsoft.compute/capacityreservationgroups’ | project id, name, location” -o json

Quota check

az vm list-usage –location \<REGION> -o table

az rest –method GET –url ‘https://management.azure.com/subscriptions/\<SUB>/providers/Microsoft.Quota/groupQuotas?api-version=2025-03-01-preview’

Throttle headers

az rest –method GET –url ‘\<RESOURCE_URL>?api-version=\<VERSION>’ –verbose 2>&1 | grep -i x-ms-ratelimit

Generic REST template

az rest –method \<GET|POST|PUT|PATCH|DELETE> –url ‘\<RESOURCE_URL>?api-version=\<VERSION>’ –body ‘\<JSON>‘

Appendix B — Common Error Codes and Resolutions

Error Code	Meaning	Resolution
CapacityReservationGroup-SharingNotEnabled	Sharing preview not enabled for subscription or API version	Request preview access through Microsoft; pin a supported api-version (try 2024-03-01 then 2024-03-01-preview); record version that works
CapacityReservationGroup-LimitReached	100-consumer (or observed) limit hit on sharingProfile	Shard CRG; update engine sharding logic; do not force additional consumers onto the same CRG
QuotaExceeded	Insufficient quota for SKU family or regional vCPU	Request quota increase in Azure Portal / Quota API before retesting; re-run dual provider and consumer checks
429 TooManyRequests	Throttle limit hit for subscription or resource provider	Wait Retry-After seconds; reduce test concurrency; capture x-ms-ratelimit headers; adjust adaptive throttle manager
AuthorizationFailed	Insufficient RBAC at the targeted scope	Verify role assignments at correct scope (CRG resource ID, not only resource group); separate reader vs mutator identities
ResourceGroupNotFound / ResourceNotFound	Resource does not exist or wrong subscription context	Verify az account show matches target subscription; confirm resource group and names
InvalidApiVersionParameter	API version not supported in this region/cloud for the feature	Try alternative preview API versions; document result per region; do not assume one version works everywhere
ZonalAllocationFailed	No capacity available in the requested zone, or zone mapping mismatch	Try alternative zone; re-run POC-06a zone translation; document as capacity or mapping constraint
ZoneMappingUnavailable		Block onboarding/deployment; rebuild

Error Code	Meaning	Resolution
	Engine/onboarding cannot resolve consumer logical zone for provider physical zone	zone_mapping_table via locations API; escalate to placement engineering
MethodNotAllowed / 404 on Quota Groups	Quota Groups API unavailable in scope (POC-30 fail)	BLOCK all quota-group engineering; escalate to Quota Owner; consider fallback architecture

Appendix C — Critical Design Constraints (Quick Card)

Carry-forward constraints from the finalized architecture. Do not weaken during testing.

ID	Constraint	Test implication
FC-06	Cross-subscription logical AZ mappings differ	POC-06a onboarding gate; zone translation mandatory
FC-08	VMSS reprovision via shared CRG in zone outage is Preview limitation	POC-VMSS-DR must run before Phase 2 VMSS DR design
FC-09	RI discount does not autoflow to consumer	POC-RI-01/02 before customer cost models
FC-11	quota groupType enforcement is preview-only	Do not depend on EnforcedGroup until GA + POC-30
FC-16	Compute baselines 250 reads/5 min; 1,200 writes/hour	POC-THROTTLE-*; adaptive manager starting config
FC-18	AKS node pool CRG change requires recreation	POC-AKS-02 documents disruption
G-14	Consumer credential model unresolved	Tier 3 blocked; POC-20 must reject
G-15	Engine mode state machine incomplete	DR automation blocked until implemented
D-REG	Geography-aware region model (v2.4): Prod ≠ NonProd always; DR distinct only in three-region geos (US); DR co-located with CVAL in two-region geos (PLC-010a); Middle East `DR_NOT_OFFERED` (DEC-001)	PF-09/PF-10 evaluate against the geography's `distribution_model`; POC-PLC-010a is the positive two-region case
T3	Tier 3 human-only in Phase 1	No automation; VMSS Tier 3 rejected

Appendix D — Cleanup Procedure

Run cleanup at the end of each test day unless resources are intentionally retained for a multi-day scenario. Order matters: remove associations before deleting reservations and CRGs.

1. List test VMs and deallocate: az vm list -g \<RG> -o table then az vm deallocate –ids \<id>
1. Disassociate VMs from CRGs (POC-05 pattern).
2. Delete test VMs: az vm delete –resource-group \<RG> –name \<VM> –yes
3. Delete test VMSS / AKS node pools created solely for POC.
4. Set Capacity Reservation capacity to 0 if required by policy, then delete: az capacity reservation delete …
5. Clear sharingProfile consumer entries that were added only for test.
6. Delete test CRGs: az capacity reservation group delete …
7. Remove test quota group memberships / groups if POC-30 created them and policy allows.
8. Verify with ARG and ARM that no orphaned test resources remain.
9. Attach cleanup evidence (commands + timestamps) to the daily execution log.

Appendix E — Sign-Off Record

Final workbook closure signatures after all applicable phase gates for the current programme stage.

Role	Name	Signature	Date	Stage (P1 / P2 / Prod)
Test Lead				
Capacity Architect				
Security				
DR Owner				
Quota Owner				
Product Owner				
Design Review Council				

Appendix F — Daily Execution Log Templates

Use one block per test day. Attach CLI transcripts and request IDs to the evidence store referenced below.

Day 1 log

Field	Entry
Date	
Engineer(s)	
Subscription context(s)	
POCs planned	
POCs completed (Pass/Fail/Blocked)	
Blockers raised (IDs)	
Resources created (names)	
Resources deleted / cleanup done (Y/N)	
Evidence location (path / ticket)	
Notes for next day	

Day 2 log

Field	Entry
Date	
Engineer(s)	
Subscription context(s)	
POCs planned	
POCs completed (Pass/Fail/Blocked)	
Blockers raised (IDs)	
Resources created (names)	
Resources deleted / cleanup done (Y/N)	
Evidence location (path / ticket)	
Notes for next day	

Day 3 log

Field	Entry
Date	
Engineer(s)	
Subscription context(s)	
POCs planned	
POCs completed (Pass/Fail/Blocked)	
Blockers raised (IDs)	
Resources created (names)	
Resources deleted / cleanup done (Y/N)	
Evidence location (path / ticket)	
Notes for next day	

Day 4 log

Field	Entry
Date	
Engineer(s)	
Subscription context(s)	
POCs planned	
POCs completed (Pass/Fail/Blocked)	
Blockers raised (IDs)	
Resources created (names)	
Resources deleted / cleanup done (Y/N)	
Evidence location (path / ticket)	
Notes for next day	

Day 5 log

Field	Entry
Date	
Engineer(s)	
Subscription context(s)	
POCs planned	
POCs completed (Pass/Fail/Blocked)	
Blockers raised (IDs)	
Resources created (names)	
Resources deleted / cleanup done (Y/N)	
Evidence location (path / ticket)	
Notes for next day	

Appendix G — Evidence Classification Guide

Every POC result must be classified so programme reporting does not overstate platform guarantees (architecture evidence hierarchy).

Classification	When to use	May support
Documented	Official Microsoft behaviour with claim-level source support	Design decisions citing platform guarantees
Observed	Executed POC with retained logs for a specific API version, region, SKU, zone	Internal tested conclusion — not a Microsoft commitment
Assumed	Workbook expected result not yet executed, or design hypothesis	Planning only; cannot close production gates alone
Derived	Logical consequence of the approved design or formulas	Internal consistency checks
Judgement	Council recommendation or risk disposition	Governance decisions, not platform facts

Preview features (CRG Sharing, Quota Groups, groupType) must never be labelled Documented as GA behaviour. POC observations of preview APIs are Observed only.

Appendix H — Related Architecture References

Cross-references into the finalized production-readiness and architecture document.

Topic	Architecture section	Workbook coverage
Executive decision / pilot scope	§1, §16, §18	Phase gates §6
Evidence hierarchy	§2	Appendix G
Assumption matrix A-01…A-20	§8	§8
FC-06 zone alignment	§5	POC-06a
FC-08 VMSS shared CRG zone outage	§6	POC-VMSS-DR
FC-09 RI discount scope	§38	POC-RI-01/02
FC-11 groupType preview	§26	POC-30
FC-16 Compute throttle baselines	§7	POC-THROTTLE-*
FC-18 AKS node disruption	§6, §33	POC-AKS-02
G-14 credentials	§11, §36, §39	POC-20, blocker log
G-15 engine mode	§29, §39	POC-18, Phase 2 gates
Tier escalation	§32	POC-18/19/20
Production entry gates PG-01…PG-10	§17	§6.4
Risk register R-01…R-44	§40	Blocker log + residual risk acceptance
Runbooks A–E	§41	Operational companion (not duplicated here)
POC validation plan	§42	This workbook (authoritative execution form)